Programski prevodioci

01 Uvod

25-26

Dunja Vrbaški

Fakultet tehničkih nauka, Novi Sad

- Šta je programski jezik?
- U čemu je razlika između jezika: C++, Java i Python
- Šta su programski prevodioci?
- Kako su se razvijali programski jezici?

- Koja je razlika između kompajlera i interpretera?
- Šta je virtuelna mašina?

- Šta je “stek frejm”?
- Šta znači kad kažemo da se nešto čuva “na steku” ili “na hipu”?
- Šta je “nešto” u prethodnom pitanju?

- Šta se desi kad napišemo **`x = 5`** ili **`str = “abc”`**
    - C
    - C++
    - C#
    - Java
    - Python
    - …

- Šta je promenljiva?
- Da li svi jezici imaju promenljive?
-
- Šta je objekat?

- Šta znači statički ili dinamički tipizirani jezik?

- Kada su dva stringa jednaka u Javi?
- Šta znači jednaka?

- Kakve sve greške možemo imati kad programiramo?

- Šta je bolje?

```
i++;
++i;

x = i++;
x = ++i;
```

- Šta znači bolje?

- Šta je bolje?
    - 5x blok
    - petlja koja se ponavlja 5x

- Koji programski jezik je najbolji?

- Prevodilac je softver. U kom programskom jeziku je napisan taj softver?

Programiranje - ideja

Programski jezik - alat

Poznavanje programskog jezika:

- sintaksa + kako se prevodi → kako se izvršavaju programi
- osnova, ideje, koncepti
- istorija, trendovi, dobre prakse

Programski prevodilac prevodi program napisan u jednom jeziku na drugi jezik.

Cilj: ono što programer razume $\rightarrow$ ono što računar razume

viši programski jezik → asembler ili mašinski jezik

viši programski jezik → međukod

viši programski jezik → drugi programski jezik

Primeri:

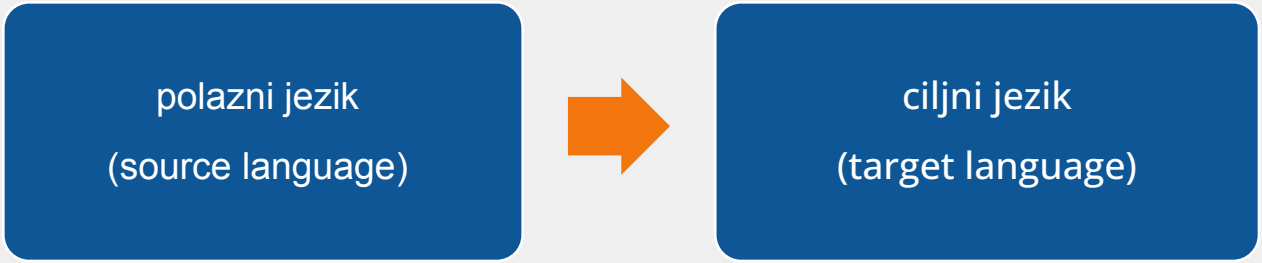
Clang/LLVM (C/C++ → IR → mašinski kod)
Emscripten (C++ → WebAssembly)
Rustc (Rust → LLVM → target)
TypeScript compiler (TS → JS)

```cpp
int testFunction(int* input, int length) {
  int sum = 0;
  for (int i = 0; i < length; ++i) {
    sum += input[i];
  }
  return sum;
}
```

```asm
testFunction(int*, int):
        test    esi, esi
        jle     .L4
        movsx   rsi, esi
        xor     eax, eax
        lea     rdx, [rdi+rsi*4]
.L3:
        add     eax, DWORD PTR [rdi]
        add     rdi, 4
        cmp     rdi, rdx
        jne     .L3
        ret
.L4:
        xor     eax, eax
        ret
```

C++

x86-64, gcc 13.2, -O2 , intel asm



Najvažniji cilj: prevesti ispravno

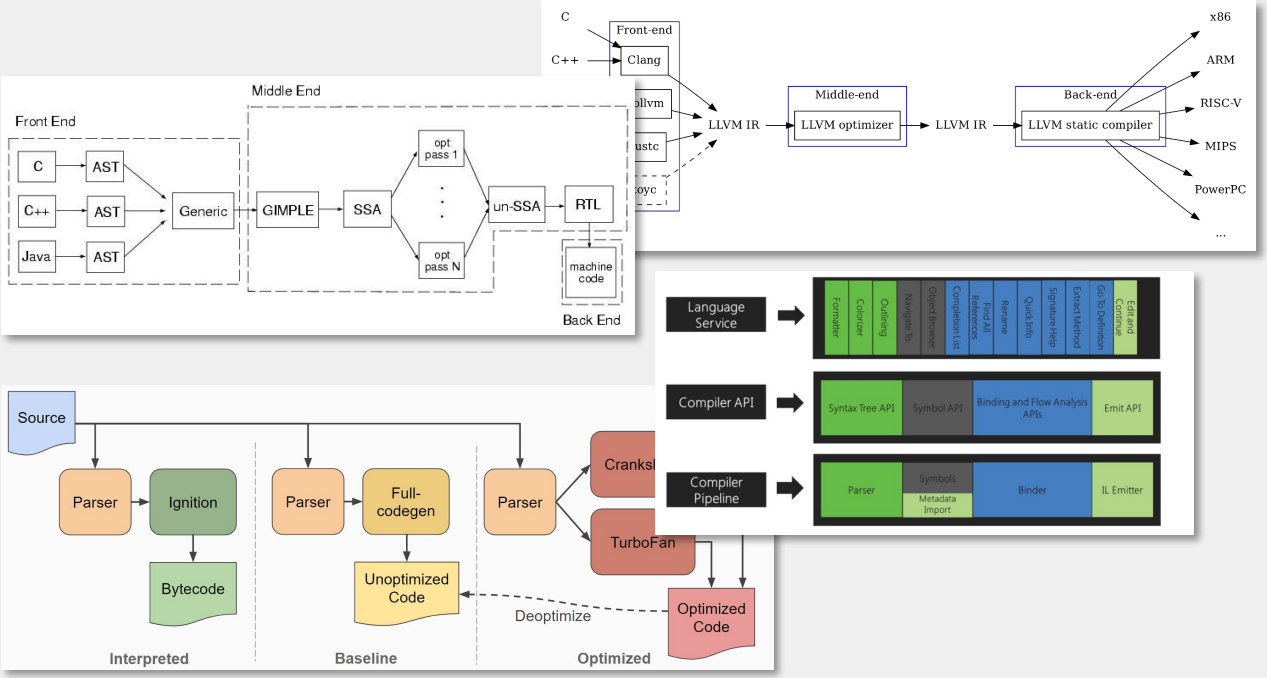
očuvati značenje
dobiti očekivano ponašanje

Najvažniji cilj: prevesti ispravno

Ostali ciljevi (značajne karakteristike prevodilaca)

- brzo izvršavanje prevođenja
- brzo izvršavanje prevedenog programa
- optimalno korišćenje resursa
- informativne poruke o greškama, oporavak od grešaka
- mogućnost proširivanja i izmena

- Kompleksan sistem koji se sastoji iz faza.
- Svaka faza – razvijana, izučavana, sa posebnim strukturama podataka i algoritmima.
- Značajan (i vidljiv) formalan uticaj teorije.



- Jedan jezik može imati više prevodilaca (implementacija)

C++20 core language features

C++20 feature	Paper(s)	GCC	Clang	MSVC	Apple Clang	EDG eccp	Intel C++	IBM XL C++	IBM Open XL C++ for AIX	IBM Open XL C++ for z/OS	Sun/Oracle C++	Embarcadero C++ Builder	Cray	Nvidia HPC C++ (ex PGI)*	Nvidia nvcc
Allow Lambda capture [=, this]	P0409R2	8	6	19.22*	10.0.0*	5.1	2021.1						11.0	20.7	12.0
VA_OPT	P0306R4	8 (partial)* 10 (partial)*	9	19.25*	11.0.3*	5.1	2021.1						11.0	20.7	12.0
			3.0 (partial)* 10	19.21*	12.0.0*	5.1	2021.1						11.0	20.7	12.0
			9	19.22*	11.0.0*	5.1	2021.1						11.0	20.7	12.0
			6	19.25*	10.0.0*	5.1	2021.1						11.0	20.7	12.0
			6	19.14*	Yes	5.0	2021.1						11.0	20.7	12.0
			6	19.0	10.0.0*	5.1	2021.1						11.0	20.7	12.0

C++23 core language features

C++23 feature	Paper(s)	GCC	Clang	MSVC	Apple Clang	EDG eccp	Intel C++	IBM XL C++	IBM Open XL C++ for AIX	IBM Open XL C++ for z/OS	Sun/Oracle C++	Embarcadero C++ Builder	Cray	Nvidia HPC C++ (ex PGI)*	Nvidia nvcc
Literal suffix for						6.5	2022.2								
						6.3	2022.2								
						6.3	2022.2								
						N/A									
							2022.2								
							2022.2								
							2022.2								
						Yes	Yes								

C++26 core language features

C++26 feature	Paper(s)	GCC	Clang	MSVC	Apple Clang	EDG eccp	Intel C++	IBM XL C++	IBM Open XL C++ for AIX	IBM Open XL C++ for z/OS	Sun/Oracle C++	Embarcadero C++ Builder	Cray	Nvidia HPC C++ (ex PGI)*	Nvidia nvcc
DR98: Removing undefined behavior from lexing	P2621R3	Yes	3.3												
DR98: Making non-encodable string literals ill-formed	P1854R4	14	14												
Unevaluated strings	P2361R6	14	17 (partial)* 18												
Adding @, $, and ` to the basic character set	P2558R2	15	Yes												
constexpr cast from void*	P2738R1	14	17												
DR20: On the ignorability of standard attributes	P2552R3		Yes												
DR11: Static storage for braced initializers	P2752R3	14													
User-generated static_assert messages	P2741R3	14	17												

https://en.cppreference.com/w/cpp/compiler_support

Vrsta	Opis	Primer
Kompajler	Prevodi ceo program u izvršni fajl	GCC, Clang
Interpreter	Izvršava „u hodu“	Python, Bash
JIT kompajler	Kombinuje oba pristupa	Java HotSpot, V8
Transpajler	Prevodi na jezik istog nivoa	TypeScript → JS
AI-kompajler	Model koji generiše i optimizuje kod	GitHub Copilot

ZABLUDE

- greške u programu != sintaksne greške
- interpreter nije “spor” kompajler
- Python se interpretira, Java se kompajlira
- IR je opcioni - danas maltene nije
- optimizacija != trikovi, formalna je stvar
- kompajler je ono što mi radimo na vežbama

Razvoj programskog jezika:

Osmisliti + definisati + implementirati

→ Softverski projekat

[osmisliti] Šta sve treba odlučiti?

- Ideja
- Opšti ciljevi: edukacija, istraživanje ili profesionalno; DSL ili opšte namene; izvršavanje programa ili međujezik...
- Specifični tehnički ciljevi: brzina prevođenja, brzina izvršavanja, sigurnost, pristup i upravljanje memorijom,...
- Razvoj: koristimo alate ili sve radimo samostalno; svaki korak zasebno ili neke spajamo; koji jezik koristimo za implementaciju;...

sve odluke povezane; često postoji kompromis

[definisati] Šta sve treba precizirati?

- Leksika jezika: skup pravila koja definišu šta sve čini jezik (azbuka i dozvoljene reči)
- Sintaksa jezika: skup pravila koja definišu ispravno formirane programske strukture (gramatika)
- Semantika jezika: skup pravila koja određuju i upravljaju značenjem jezika
- Strukture podataka i alg.: kreiraju se u različitim fazama prevođenja (tabele, stabla...)
- Obrada grešaka: gde i na koji način se vrši obrada grešaka
- Međujezik: poznavati postojeći ili formirati novi
- Optimizacije: obavezne i dodatne
- Jezici i alati: koje tehnologije koristiti
- Arhitektura rešenja: procesi, integracije, automatizacija, build alati, testiranje

→ funkcionalan sistem

[implementirati] Šta sve treba poznavati?

- Strukture podataka: stek, red, heap, heš tabele, stabla, grafovi,...
- Algoritmi: obilasci stabla/grafova, rekurzivni spust, bojenje grafova,...
- Teorija formalnih jezika: regularni jezici, kontekstno osetljive/slobodne gramatike,...
- Teorija automata: konačni automati (deterministički, nedeterministički), potisni automati,...
- Programski jezici: jezik za koji pravimo prevodilac; jezik u kom implementiramo prevodilac; ciljni jezik
- Arhitektura računara, operativni sistemi: zbog ciljnog jezika, izvršavanja
- Metode testiranja
- Razvojni alati
- Terminologija

Jednostavno ili ne?

Primitivni prevodilac, univerzitet, hobby jezik – nije jako teško

Prevodilac za industriju, DSL – znatno teže

Prevodilac opšte namene - prilično ozbiljno (standardizacije)

Teorija – matematika, formalizmi → vidljivost značajna i na osnovnim nivoima

Puno različitih povezanih koraka/delova → lako može postati nepregledno/neuredno

Proširivost

Korisnici su programeri!

Bagovi i problemi u kompajleru → veliki problem

Programer očekuje: brzo + pouzdano + jednostavno za korišćenje

U opštem slučaju ne obraća pažnju na kompajler, rešava SVOJ problem.

Korisnici su, u tom smislu, zahtevniji nego korisnici ostalih softverskih rešenja.

Da bi se (šire) prihvatio prevodilac obično očekuju i neku biblioteku (core, standard) i alate.

CILJEVI KURSA - OPŠTI

Da znate sigurno da odgovorite na pitanja vezana za dizajn i implementaciju programskog jezika.

Da uspešno birate i koristite bilo koji jezik.

CILJEVI KURSA - SPECIFIČNI

Napraviti kompajler koji radi.

Uvideti šta su osnovni zadaci svakog prevodioca.

Shvatiti šta su problemi i neke pristupe u rešavanju tih problema.

Upoznati malo bolje neke PJ (C, C++, Java, Python)

Programski jezici u modernim okvirima

Zašto učimo ako već svi prevodioci postoje?

ne mora svako da zna napiše kompajler

- prethodno navedeno → Zaokružiti priču o programskim jezicima
- C++ 26?
- CS osnovna znanja
- PL za specifične domene
- Znanja, metode, tehnike, koncepti, teorija – primena u drugim oblastima
- Izazovi u razvoju komplikovanijeg softvera / metodološki pristup
- Obrada teksta i govora (prirodni jezik)
- Nova zainteresovanost u oblasti

Možda, za većinu, najvažnije:

Osnova svih modernih alata: linteri, transpajleri, IDE alati, JIT, AI asistenti...
Koriste iste faze obrade (metode, znanja) kao i klasični prevodioci.